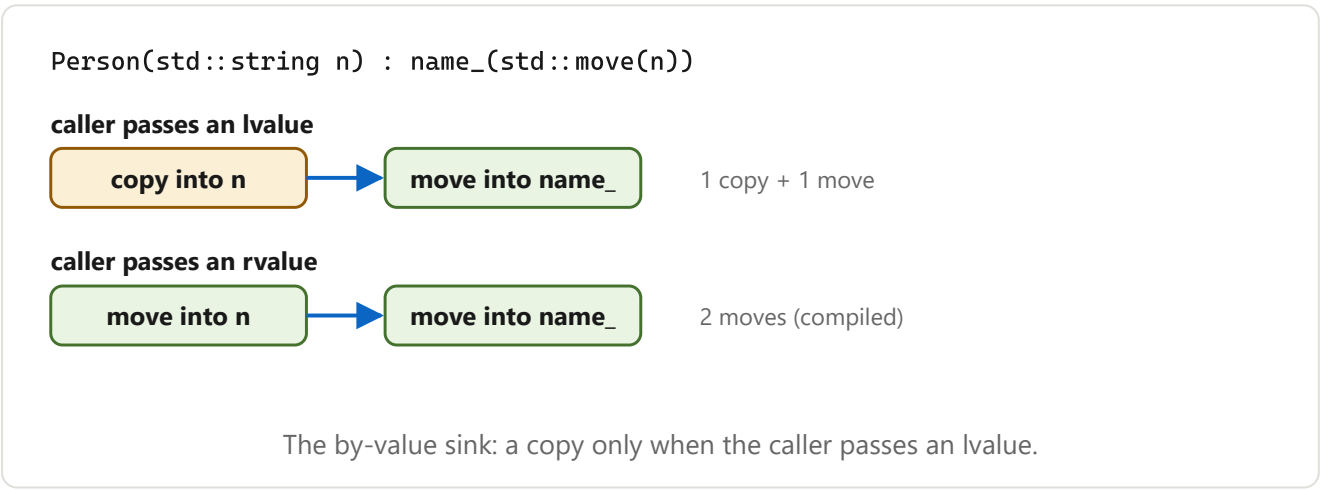
Sink Parameters and Move Traps

Taking a parameter that you will store: by value and move versus overloads, the traps that turn a move into a copy, and `push_back` versus `emplace_back`. Assumes Returning Objects and Copy Elision.

Taking a parameter you will store

A function that **stores** its argument (usually a constructor) wants to copy an lvalue and move an rvalue. The simplest way to get both: take the parameter **by value**, then move it into place.



The by-value sink: a copy only when the caller passes an lvalue.

```cpp
class Person {
    std::string name_;
public:
    // by value, then move into the member
    explicit Person(std::string n) : name_(std::move(n)) {}
};
std::string s = "Ana";
Person a(s);                 // copies s into n, moves n into name_
Person b(std::string("Bo")); // moves twice, never copies
```

Compiled with a counting type: one lvalue call and one rvalue call together made **1 copy and 2 moves**.

By value, overloads or const reference?

Three ways to write a constructor that stores its argument:

STYLE	LVALUE ARGUMENT	RVALUE ARGUMENT	FUNCTIONS TO WRITE
by value, then `std::move`	1 copy + 1 move	2 moves	one
overloads `const T&` and `T&&`	1 copy	1 move	two
only `const T&`	1 copy	1 copy	one (slower for temporaries)

- By value costs **one extra move** compared with two overloads. For cheap-to-move types that is the best trade for the saving in code.
- Two overloads are fastest, and worth it when a move is expensive or the function is on a hot path (compiled: 1 copy, 1 move).
- Do **not** use by-value sinks for **setters**: assigning into an existing member can reuse its storage, while a by-value parameter always allocates first.
- For types that are cheap to copy (`int` , a pointer), just take them by value and do not move.

Move traps

The move operations are only used when everything lines up. These are the usual ways they are not:



A const rvalue reference does not bind to T&&.

TRAP	WHAT HAPPENS
`std::move` on a `const` object	**copies**: a `const T&&` cannot bind to `T&&` (compiled: 1 copy, 0 moves)
a type with no move operations	silently copies
using an object after `std::move`	valid but unspecified; only destroy or assign to it
`std::move` on a forwarding reference parameter	moves an lvalue the caller still wanted: use `std::forward`
moving inside a loop	the second iteration sees a moved-from object
`return std::move(local);`	blocks elision: an extra move

> **MOVE ONLY THE LAST USE** After `auto b = std::move(a);` treat `a` as empty. Do not assume a move happened: Effective Modern C++ Item 29 says to assume moves may be absent, not cheap and not used.

`push_back`, `emplace_back` and moving into containers

Adding to a container is the most common place to move or copy.

CALL	WHAT IT DOES
`v.push_back(lvalue)`	copies the object into the container
`v.push_back(std::move(x))` or `push_back(temp)`	moves it in
`v.emplace_back(args ... )`	builds the element in place from the arguments, with no temporary

```cpp
std::vector<Loud> v;
v.reserve(8);
Loud a(1);
v.push_back(a);                 // 1 copy
v.push_back(Loud(2));           // 1 move
// 0 copies, 0 moves: constructed in the vector
v.emplace_back(3);
```

- For a type that is cheap to build from its arguments, `emplace_back` saves the temporary and the move.
- If you already have an object, `push_back(std::move(obj))` is just as good.
- Both can reallocate, and reallocation moves or copies the existing elements (see Writing Move Operations).

Move-only sinks

For types that cannot be copied (`unique_ptr`, `std::thread`, a file handle) a by-value parameter is the natural sink: the caller must write `std::move`, so the transfer of ownership is visible.



By-value sinks make ownership transfer visible at the call site.

```cpp
// takes ownership
void take(std::unique_ptr<int> p) { std::cout << *p; }

auto p = std::make_unique<int>(7);
// take(p);                     // error: unique_ptr cannot be copied
take(std::move(p));             // p is now null
take(std::make_unique<int>(8));  // a temporary: no std::move needed
```

PARAMETER	MEANING FOR A MOVE-ONLY TYPE
`std::unique_ptr<T>`	sink: the function owns it
`std::unique_ptr<T>&`	the function may reseat the caller's pointer
`const std::unique_ptr<T>&`	pointless: pass `T&` or `T*` to just use the object
`std::unique_ptr<T>&&`	possible, but the by-value form is clearer

The same pattern is used by `std::thread`'s constructor and by containers of `unique_ptr`: `v.push_back(std::move(p))`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Taking a parameter you will store	C++ Move Semantics	4.3.1 to 4.3.4 Initializing members with move semantics	79 to 90
By value, overloads or const reference?	Effective Modern C++	Item 41, Consider pass by value for copyable, cheap-to-move parameters that are always copied	299 to 310
	C++ Move Semantics	4.3.5 to 4.3.6 Member initialization summary; always pass by value and move?	90 to 93
Move traps	Effective Modern C++	Item 29, Assume move operations are not present, not cheap, and not used	221 to 225
	C++ Move Semantics	4.1 to 4.2 Avoid objects with names; avoid unnecessary std::move	77 to 79